



Apache Flink

Apache Flink

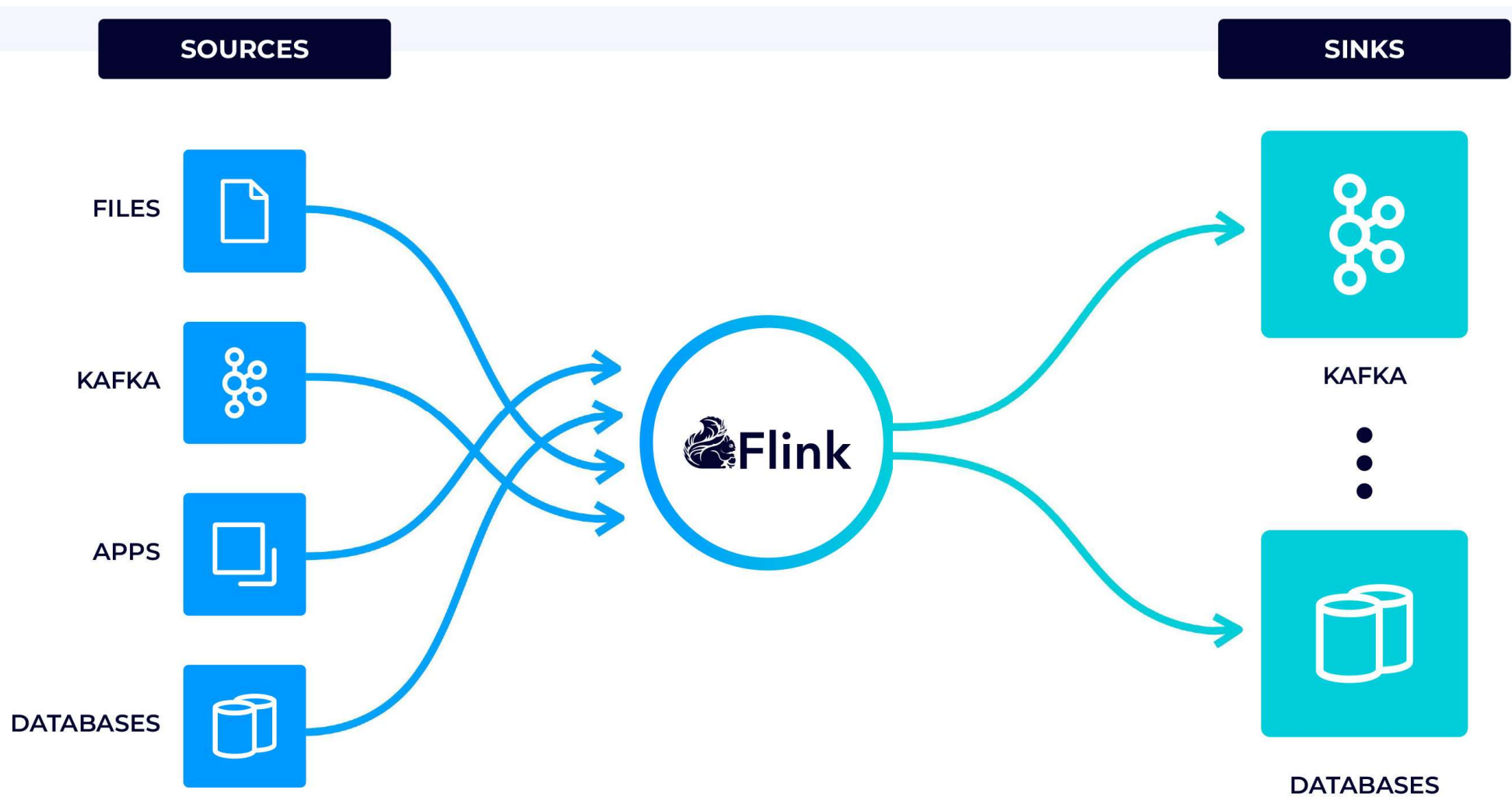
de facto standard for real-time stream processing

Consumes input streams produced by event sources

Produces output streams that are consumed by sinks

The sinks store results and make them available for further processing.

Stream Processing with Flink



Common use cases

Streaming data pipelines	Real-time analytics	Event-driven applications
Continuously ingest, enrich, and transform data streams, loading them into destination systems for timely action (vs. batch processing).	Continuously produce and update results which are displayed and delivered to users as real-time data streams are consumed.	Recognize patterns and react to incoming events by triggering computations, state updates, or external actions.
Some examples include: •Streaming ETL •Data lake ingestion •Machine learning pipelines	Some examples include: •Ad campaign performance •Usage metering and billing •Network monitoring •Feature engineering	Some examples include: •Fraud detection •Business process monitoring and automation •Geo-fencing

What makes Flink special?

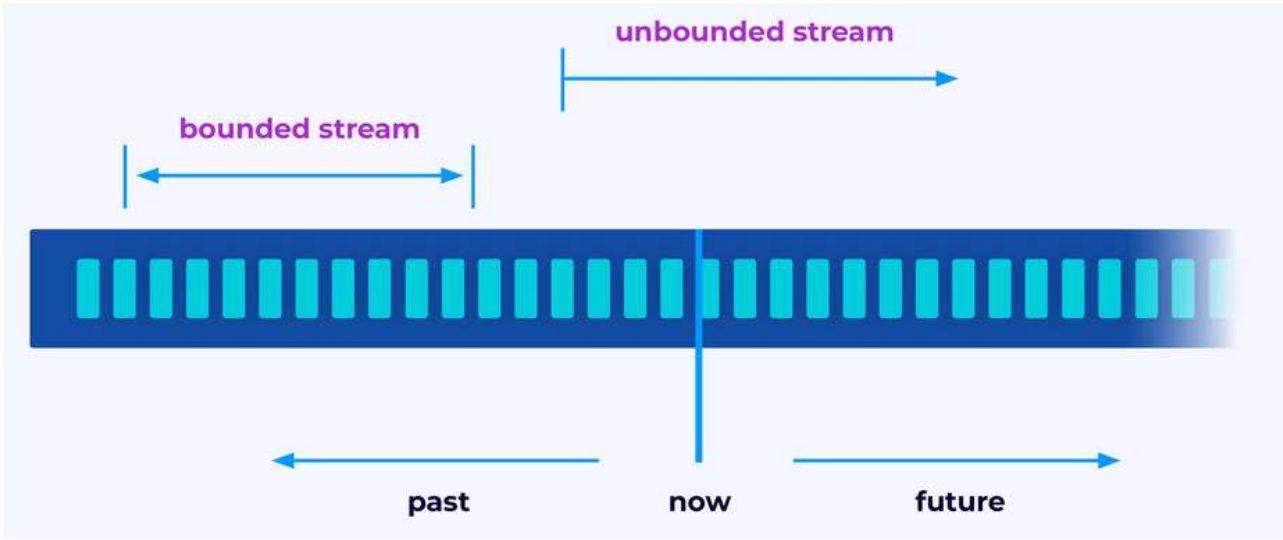
Robust support for data streaming workloads.

Strong guarantees of exactly-once correctness and failure recovery.

Support for Java, Python, and SQL

Unified support for both batch and stream processing.

Streams



Flink application

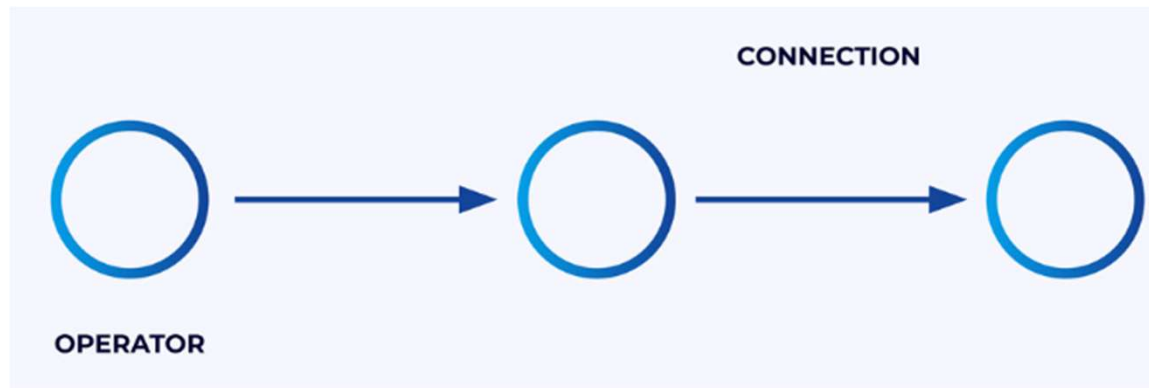
A data processing pipeline

Events flow through this pipeline

They are operated on at each stage by code we write

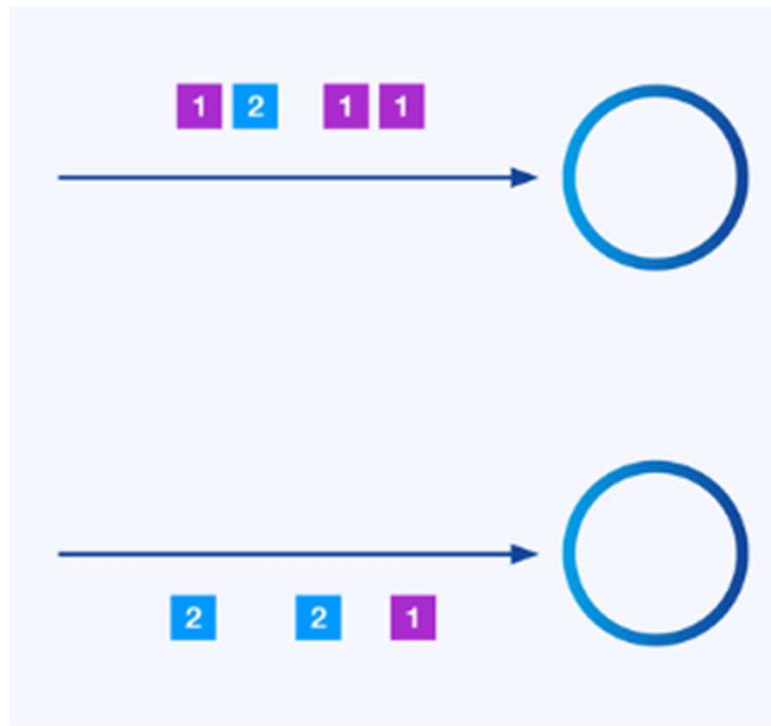
We call this pipeline the job graph

The nodes of this graph are called operators



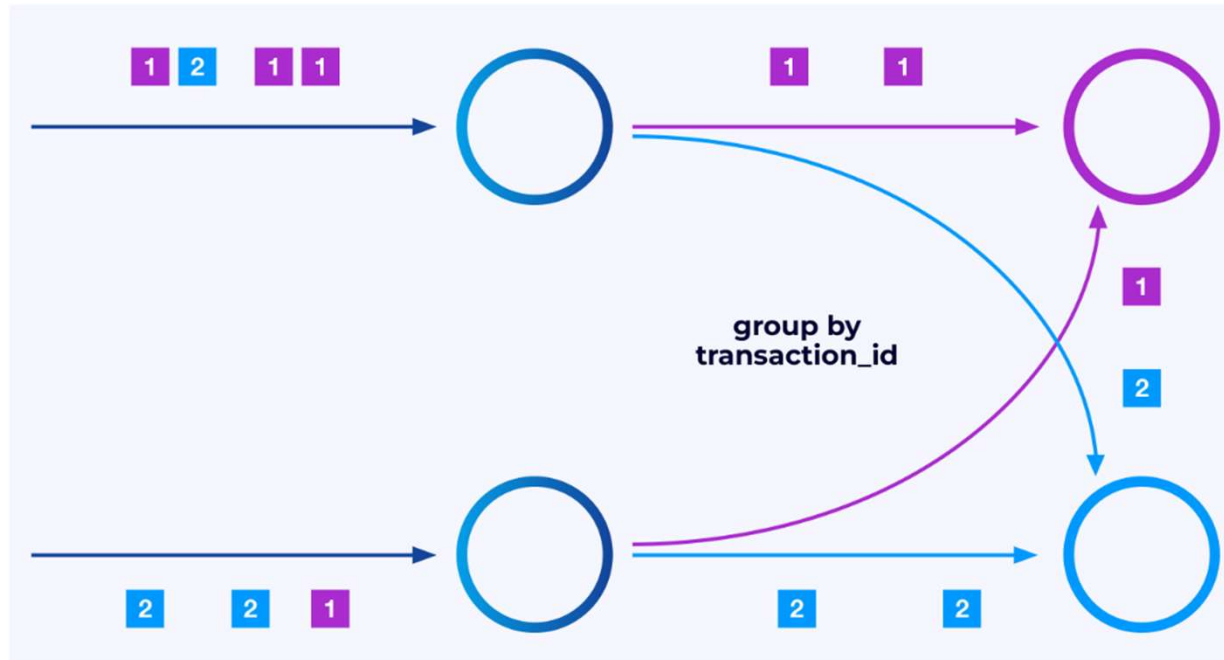
Parallel processing

- Each operator can have many parallel instances
 - Each operating independently on some subset of the events.



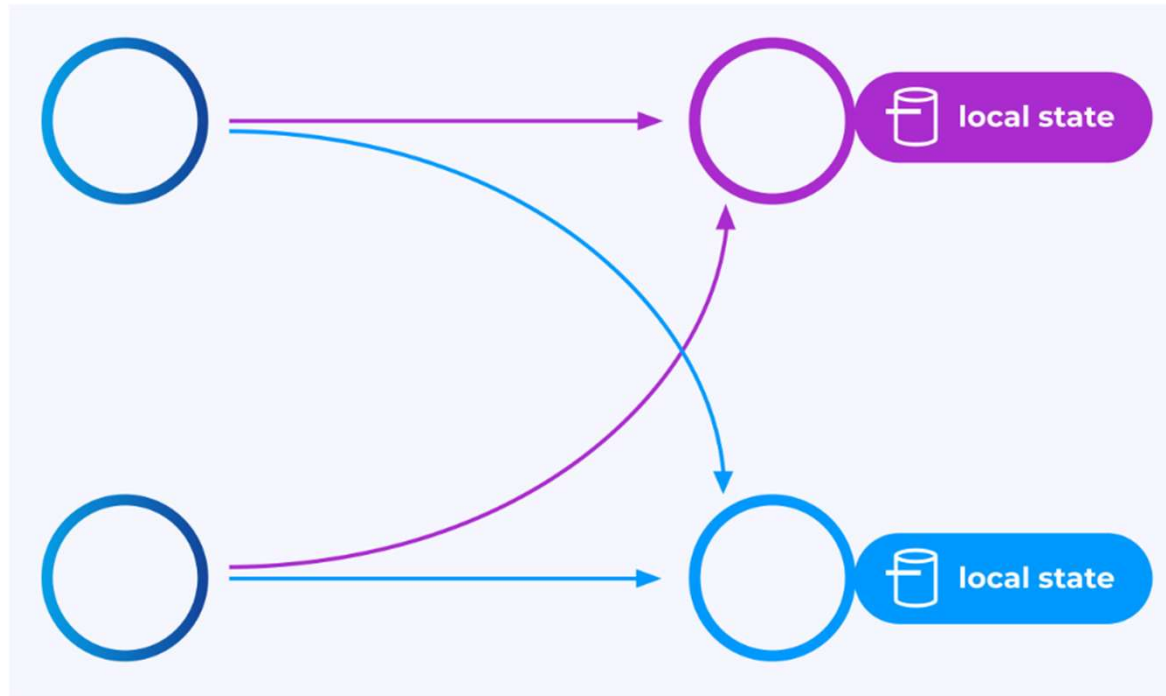
Partitioning

- Events are grouped together according to application-specific logic
- For example, if we are processing financial transactions
 - Every event for any given transaction to be processed by the same thread.



State

- Operators can use distributed key/value state store to persist
- The state for each key is local to a specific instance of an operator
 - Cannot be accessed from anywhere else



Checkpointing for failure recovery

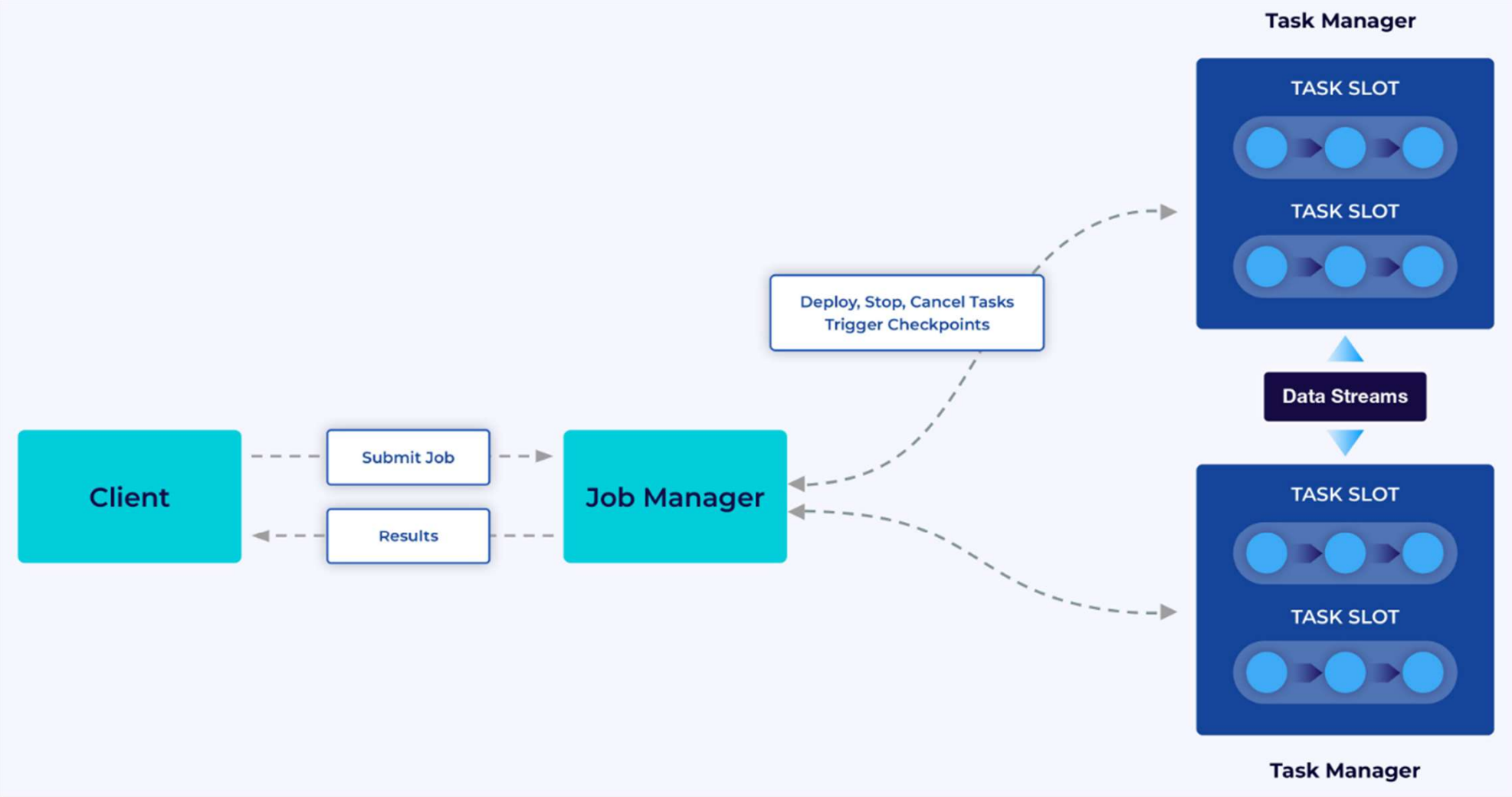
Flink is able to provide effectively exactly-once guarantee

It does this by taking periodic, global, self-consistent snapshots of all the state.

These snapshots, created and managed automatically by Flink, are called checkpoints.

Recovery involves rolling back to the state captured in the most recent checkpoint, and performing a global restart of all of the operators from that checkpoint

Flink system architecture



Flink developer experience

DataStream API

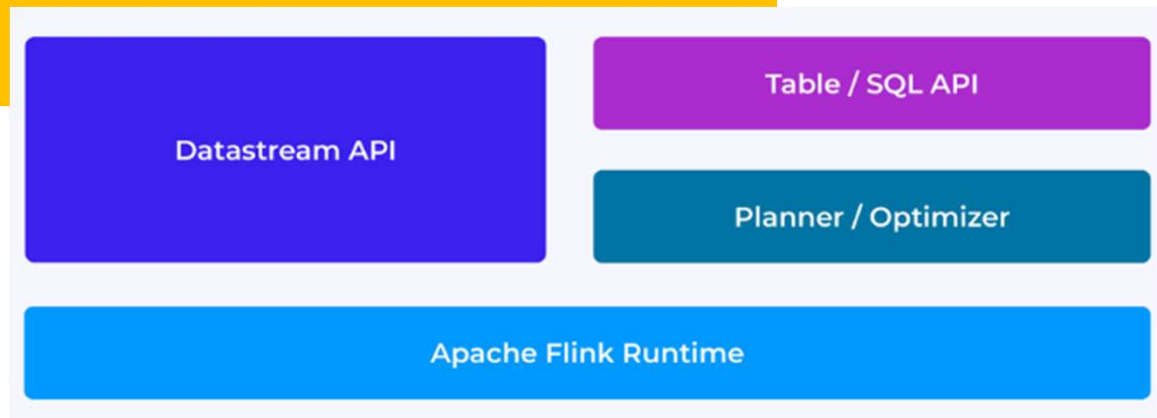
- Manually build the job
- Handling each event with functions and managing low-level details like state and timers.

Table/SQL API

- Write queries on tables, focusing on business logic, while Flink's optimizer handles low-level details efficiently.

Interoperability

- The two APIs can be combined



Streaming data pipelines

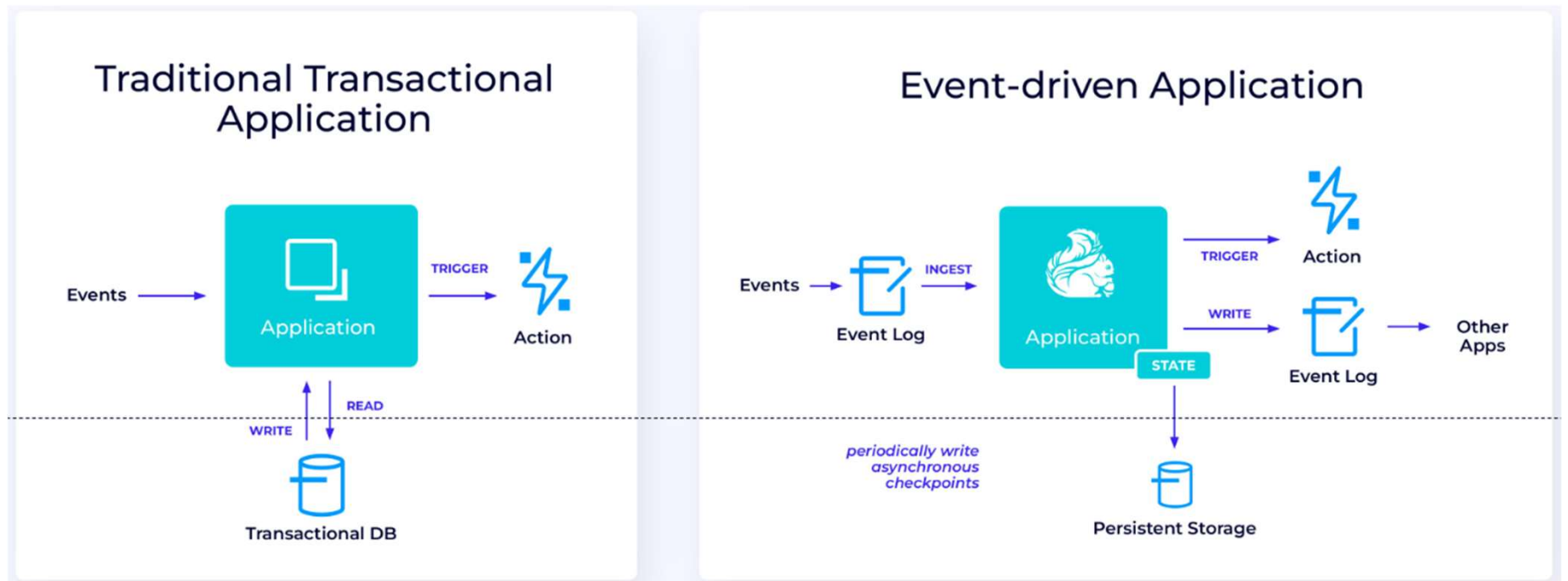
Batch ETL



Streaming ETL



Event-driven applications



Thanks